

Audience: Leadership team **Duration:** ~5–7 min + Q&A **Goal:** Show how a requirement becomes a validated, review-ready test-case workbook — safely.

The one line to land: "PS-TestAuthoring is a **document-driven** AI QA analyst. The **Knowledge** folder is the single source of truth, the **Skills** folder runs the workflow, and **Python validators** are hard gates. The model proposes — the validators dispose. If it doesn't pass, it isn't delivered."

1 · OPEN

~45 sec

Say: "Turning Jira stories into manual test cases is slow, repetitive, and inconsistent across our PS projects. This assistant does that first draft in hours instead of days — while keeping a human as the final approver. What makes it enterprise-safe isn't the AI being clever; it's that every output must pass deterministic checks before anyone sees it."

Point to the chart: left = inputs, middle = the pipeline, right = the delivered Excel workbook. Bottom = what every file in the project does.

2 · WALK THE FLOW — PROMPT TO OUTPUT

~2.5 min · the numbered stages

1 INTENT Classify the ask; confirm if unclear CP-1	2 ACQUIRE Fetch all sources read-only; strip PII	3 REQ CHECK Source real & enough? CP-2	4 ANALYZE Extract ACs, rules, actors	5 PLAN Coverage: pos / neg / edge	6 DESIGN Steps + measurable results	7 VALIDATE QA rulebook; self-correct	8 ASSEMBLE Build + validate workbook
--	--	--	--	---	---	--	--

How to narrate it: "A user types 'generate test cases for SAMP-166'. The assistant **classifies** the request, **pulls** the Jira story, its Acceptance-Criteria field and any linked Confluence, then **checks the source is real** before spending effort. It **breaks the requirement down**, **plans** positive, negative and edge coverage, then **writes** atomic steps with measurable expected results. Finally it **validates** against our QA rulebook and **builds the Excel workbook** — which a validator script inspects before it's returned."

Key phrase to repeat: each stage is governed by one Knowledge document — that's why output is consistent across every project and every person who runs it.

3 · WHY IT'S TRUSTWORTHY

~1 min

- ▶ **Gate 1 — Self-review:** a cheap pass that catches obvious gaps first.
- ▶ **Gate 2 — Validation Engine:** the QA rulebook — 100% AC coverage, negative+edge ≥ 40%, traceability, measurable results.
- ▶ **Gate 3 — validate_workbook.py: deterministic code**, not opinion — checks schema, IDs, coverage ledger. Fails ⇒ never delivered.
- ▶ **2 human checkpoints:** ambiguous ask (CP-1) or missing/contradictory source (CP-2) → it asks, never invents.

4 · THE ARCHITECTURE IN ONE BREATH

- ▶ **Knowledge/** — the "what & why". One document owns each concept, so nothing drifts or contradicts.
- ▶ **Skills/** — the "how it runs": the workflow state machine plus ID ledgers that keep test-case IDs unique across every workbook.
- ▶ **Validators (Python)** — the enforcement layer that makes "review-ready" a fact, not a claim.
- ▶ **Output** — one .xlsx per run: Master Summary with coverage %, feature sheets, plus a coverage sidecar — **pending human QA review**.

5 · NUMBERS & STATUS TO CITE

- ▶ **Production today:** one skill — TestCaseAuthoring. Others (automation scripts, API tests, test data) are on the roadmap as **Planned**.
- ▶ **Design target:** 100% acceptance-criteria coverage, proven at design time and shown in the Master Summary; test design time cut from **days to hours**.
- ▶ **Honest framing:** output quality tracks input quality — a thin requirement yields a low-confidence draft with open points flagged, not invented behaviour. That's a feature.

6 · LIKELY QUESTIONS — QUICK ANSWERS

"Does it change our Jira / Confluence?"	No — strictly read-only via the Atlassian MCP. It never writes back.
"Can it just skip validation to go faster?"	No. Validation is a guardrail; it cannot be turned off, even on request.
"What about data security / PII?"	PII and secrets are minimised at ingest; requirement text is processed by a cloud LLM — governed by DATA_HANDLING.
"Is the AI making up test cases?"	No — it tests only what's documented and flags gaps as open points. Humans approve before use.
"Why Excel and not push to Zephyr?"	Universal, no tool lock-in. Direct test-tool import is on the roadmap.

7 · CLOSE — THE ASK

Say: "It's live for one capability and proven on real epics. The ask is to **standardise this as the PS QA authoring method** and prioritise the next capabilities — flat export for Zephyr import, and test-data generation. Same methodology, every project, with a human always in the loop."